

SESSION 01-03

JAVASCRIPT

Fundamentals | Sessions 1-3

Concept architecture, dependencies, ASCII maps and practice patterns

SESSION 1

Variables / Data Types / Input

var let const | declaration | initialization |

SESSION 2

Operators / Expressions / Logic

arithmetic | assignment | comparison | logical

SESSION 3

Math / Spread / Conditionals

Math methods | spread | if/else | validation |

Structured as an original concept map - not a copy of the reference tree.

How to Read This Document

A layered map rather than a conventional lesson-by-lesson note set

DOCUMENT MODEL

JAVASCRIPT FUNDAMENTALS

```
|  
+-- SESSION 1 -> values, names, types and input/output  
|  
+-- SESSION 2 -> operations, comparisons and decisions  
|  
+-- SESSION 3 -> reusable math tools, data expansion and branching
```

Cross-session idea:

data -> operator -> condition -> action

Design principle

Each major concept is isolated into its own visual block. ASCII trees are used for structure; short prose is used only to explain what the tree means.

Source boundary

This document summarizes the concepts actually present across Sessions 1-3. Topics shown in the sample reference but not taught in the supplied sessions, such as Symbol, hoisting and the Temporal Dead Zone, are intentionally not added as session content.

Master Concept Map

The complete Sessions 1-3 architecture at one glance

MASTER ASCII TREE

JAVASCRIPT FUNDAMENTALS

```
|
+-- SESSION 1: VALUES AND STATE
|   +-- Variables
|       |   +-- var / let / const
|       |   +-- declaration / initialization
|       |   +-- redeclaration / reassignment
|   +-- Data types
|       |   +-- String / Number / Boolean
|       |   +-- Undefined / Null / BigInt
|       |   +-- Array / Object
|   +-- Input and output
|       +-- prompt() -> Number()
|       +-- console.log() -> typeof
|
+-- SESSION 2: OPERATIONS AND EXPRESSIONS
|   +-- Arithmetic -> + - * / % **
|   +-- Mutation -> ++ --
|   +-- Assignment -> = += -= *= /=
|   +-- Comparison -> == === != !== > < >= <=
|   +-- Logic -> && || !
|   +-- Ternary -> condition ? a : b
|
+-- SESSION 3: TOOLS AND CONTROL FLOW
    +-- Math -> random floor sqrt pow min max
    +-- Spread -> ...array -> function arguments
    +-- Branching -> if / else if / else
    +-- Applied logic -> validation, ranges, access, grading
```

Session 1 - Variables

How JavaScript introduces named state

SESSION 1 ASCII MAP

SESSION 1

|

+-- VARIABLES

| +-- var

| +-- let

| +-- const

|

+-- VARIABLE LIFECYCLE

| +-- declaration

| +-- initialization

| +-- declaration + initialization

| +-- redeclaration

| +-- reassignment

|

+-- CORE RULES

+-- var -> redeclaration allowed; reassignment allowed

+-- let -> reassignment allowed; same-scope redeclaration not allowed

+-- const -> initialize at declaration; reassignment not allowed

Mental model

Session 1 begins with state: a program needs named values, then needs clear rules for how those values are created and changed.

Session 1 - Data Types

Classifying the values a program works with

SESSION 1 ASCII MAP

DATA TYPES

```
|  
+-- PRIMITIVE  
|   +-- String -> textual data  
|   +-- Number -> integer / floating-point value  
|   +-- Boolean -> true / false  
|   +-- Undefined -> declared, no assigned value  
|   +-- Null -> intentional absence of value  
|   +-- BigInt -> very large integers with n suffix  
|  
+-- NON-PRIMITIVE / REFERENCE  
|   +-- Array -> ordered collection  
|   +-- Object -> key/value properties
```

OBSERVATION

```
+-- arrays and objects are the structured values introduced in Session 1
```

Important distinction

The session teaches a useful beginner classification: primitive values versus object/reference types. It does not attempt to reduce all JavaScript memory behavior to one slogan.

Key examples

```
String -> "Hello" | Number -> 56 or 12.43 | Boolean -> true | BigInt -> 123n | Array -> [1,2,3]  
| Object -> { id: 4 }
```

Session 1 - Input and Output Pipeline

Turning user input into a typed result

SESSION 1 ASCII FLOW

USER INPUT / OUTPUT FLOW

```
|
+-- prompt("Enter value")
|   |
|   +-- browser input is normally text
|
+-- Number(prompt(...))
|   |
|   +-- converts numeric text into a number
|
+-- arithmetic / operator work
|
+-- console.log(...)
|   |
|   +-- display the result
|
+-- typeof value
|   |
|   +-- inspect the resulting type
```

Canonical example

Two values are collected with `prompt()`, converted with `Number()`, added, and checked with `typeof`. This creates the first bridge between input, type conversion and computation.

Session 1 outcome

By the end of the session, the student can create values, classify values, take simple input, convert it, perform basic arithmetic and inspect results.

Session 2 - Operator Architecture

How values become expressions and results

SESSION 2 ASCII MAP

SESSION 2: OPERATORS & LOGIC

```
|
+-- ARITHMETIC
|   +-- binary: + - * / % **
|   +-- unary mutation: ++ --
|
+-- ASSIGNMENT
|   +-- = += -= *= /=
|
+-- COMPARISON
|   +-- equality: == ===
|   +-- inequality: != !==
|   +-- relational: > < >= <=
|
+-- LOGICAL
|   +-- AND: &&
|   +-- OR: ||
|   +-- NOT: !
|   +-- short-circuit evaluation
|
+-- TERNARY
|   +-- condition ? trueResult : falseResult
```

Core idea

Session 2 is about transforming values with operators and then using the results of those expressions to reason about program state.

Session 2 - Arithmetic and Assignment

Mutating numeric state with concise expressions

SESSION 2 ASCII MAP

ARITHMETIC + ASSIGNMENT

```
|  
+-- a + b -> addition  
+-- a - b -> subtraction  
+-- a * b -> multiplication  
+-- a / b -> division  
+-- a % b -> remainder  
+-- a ** b -> exponentiation  
|  
+-- ++x / --x -> change first, then use  
+-- x++ / x-- -> use first, then change  
|  
+-- x += y -> x = x + y  
+-- x -= y -> x = x - y  
+-- x *= y -> x = x * y  
+-- x /= y -> x = x / y
```

Trace mindset

Pre/post operators require tracking two things separately: the value produced by the expression and the variable value after the operator executes.

Example

`let x = 3; console.log(++x, x++);` -> first expression produces 4 and leaves x at 4; second produces 4 and leaves x at 5.

Session 2 - Comparison and Logic

Turning values into Boolean decisions

SESSION 2 ASCII MAP

COMPARISON -> LOGICAL DECISION

```
|  
+-- equality  
|   +-- 22 == "22" -> true (loose comparison)  
|   +-- 22 === "22" -> false (strict comparison)  
|  
+-- relational  
|   +-- > < >= <=  
|  
+-- inequality  
|   +-- != / !==  
|  
+-- logical combination  
    +-- A && B -> both conditions  
    +-- A || B -> at least one condition  
    +-- !A -> reversed Boolean result
```

DECISION RESULT

```
+-- true  
+-- false
```

Type note

The source explicitly contrasts loose equality with strict equality and shows how logical operators combine Boolean conditions.

Short-circuit idea

For `&&`, a false result can make the final result known before the right side is needed.
For `||`, a true result can make the final result known early.

Session 2 - Ternary and Expression Thinking

Condensing a two-way decision into an expression

SESSION 2 ASCII MAP

TERNARY OPERATOR

```
|  
+-- condition  
|  
+-- ? trueResult  
|  
+-- : falseResult
```

MENTAL MODEL

```
condition ? valueIfTrue : valueIfFalse
```

EXAMPLE

```
age >= 18 ? "You can Vote" : "Not eligible"
```

RELATION TO SESSION 3

```
+-- Session 2 -> ternary expresses a compact two-way decision  
+-- Session 3 -> if / else if / else handles multi-branch control flow
```

Expression mindset

A ternary expression evaluates to a value. That makes it useful when a program needs to choose one of two values directly.

Bridge forward

Session 3 builds on this decision-making idea with full conditional control flow and validation branches.

Session 3 - Math Object

Reusable built-in mathematical utilities

SESSION 3 ASCII MAP

SESSION 3: MATH OBJECT

```
|  
+-- Math.random()  
|   +-- pseudo-random floating-point value in [0,1)  
|  
+-- Math.floor()  
|   +-- rounds downward  
|   +-- used with random() for 0..9 integers  
|  
+-- Math.sqrt(x)  
|   +-- square root  
|  
+-- Math.pow(base, exponent)  
|   +-- power calculation  
|   +-- source notes equivalence with base ** exponent  
|  
+-- Math.min(...)  
|   +-- smallest argument  
|  
+-- Math.max(...)  
|   +-- largest argument
```

Mental model

Math is presented as a built-in toolbox. It does not create values by itself; it provides reusable methods that operate on numeric inputs.

Session 3 - Spread Operator

Expanding an array into individual arguments

SESSION 3 ASCII MAP

SPREAD WITH FUNCTIONS

```
|  
+-- array  
|   +-- [10, 20, 30, 32, 44, 2, 1, 5, 7, 4]  
|  
+-- Math.max(array)   -> not the same as many arguments  
|  
+-- Math.max(...array)  
|  
|   +-- 10, 20, 30, 32, 44, 2, 1, 5, 7, 4  
|       become individual arguments
```

CONCEPT

`...array` -> expand iterable elements into the function call

Why this matters

Session 1 introduced arrays as ordered collections. Session 3 reuses that array knowledge and connects it to function argument requirements through the spread operator.

Cross-session connection

Array -> spread -> `Math.max` / `Math.min` -> numeric result. This is one of the clearest examples of a later session building directly on an earlier data type.

Session 3 - Conditional Control Flow

From single decisions to multi-branch program flow

SESSION 3 ASCII MAP

CONDITIONAL CONTROL FLOW

```
|
+-- if (condition)
|   |
|   +-- run branch when condition is true
|
+-- else if (nextCondition)
|   |
|   +-- test another branch
|
+-- else
|   |
|   +-- fallback branch
```

VALIDATION PATTERN

```
+-- reject invalid input first
+-- evaluate valid range / rule
+-- select output branch
```

Control-flow idea

The ternary operator in Session 2 handles a compact two-way choice. Session 3 introduces a full branching structure for multiple conditions, validation and workflow rules.

Sessions 1-3 - Control Flow Evolution

How decision-making becomes more expressive across the sessions

CROSS-SESSION ASCII MAP

CONTROL FLOW EVOLUTION

```
|
+-- SESSION 1
|   +-- values + input + output
|       |
|       +-- create the data that decisions will use
|
+-- SESSION 2
|   +-- comparison -> Boolean result
|   +-- logical operators -> combine conditions
|   +-- ternary -> compact two-way decision
|
+-- SESSION 3
|   +-- if / else if / else
|   +-- validation rules
|   +-- nested branches
|   +-- workflow simulation
```

Key progression

The sessions move from representing information, to manipulating information, to making program decisions from that information.

Practical formula

value -> operator -> condition -> branch -> output

Sessions 1-3 - Data and Expression Flow

A combined pipeline from input to final behavior

CROSS-SESSION ASCII FLOW

COMBINED DATA PIPELINE

|

INPUT

|

+-- prompt() -> text

|

+-- Number() -> number

|

VALUE STORAGE

|

+-- var / let / const

|

DATA STRUCTURE

|

+-- primitive values

+-- array / object

|

OPERATIONS

|

+-- arithmetic / assignment

+-- Math methods / spread

|

DECISION

|

+-- comparison / logical / ternary

+-- if / else if / else

Why this map matters

This is the common thread across all three sessions: values are introduced, transformed, tested and finally used to produce visible program behavior.

Sessions 1-3 - Concept Dependencies

Which earlier concepts are reused by later concepts

REUSE MAP

DEPENDENCY MAP

```

|
Session 1: ARRAY
|
+-- Session 3: spread (...array)
    |
    +-- Math.max(...array)
    +-- Math.min(...array)

Session 1: NUMBER + %
|
+-- Session 2: arithmetic / comparison
|
+-- Session 3: divisibility practice

Session 2: && / || / === / %
|
+-- Session 3: validation, access and grading

Session 2: ternary
|
+-- Session 3: ticket type / compact choices
  
```

Pattern to notice

Later exercises are not isolated tricks. They combine earlier concepts: numbers + %, comparisons + &&, arrays + spread, and conditions + logical expressions.

Session 3 Practice - Problem Families

A concept map of Q3-Q20 from the supplied session

SESSION 3 PRACTICE ASCII MAP

PRACTICE FAMILY MAP

```
|
+-- INPUT VALIDATION
|   +-- Q3 age must be positive
|   +-- Q9 age categories with boundary checks
|   +-- Q10 numeric range 10..50
|
+-- COMPARISON / SELECTION
|   +-- Q4 largest of two (ternary)
|   +-- Q5 largest of three (Math.max)
|
+-- DIVISIBILITY
|   +-- Q6 divisible by 3 and 5
|   +-- Q7 divisible by 5 and 10
|   +-- Q13 positive and even
|   +-- Q14 negative or odd
|
+-- CLASSIFICATION
|   +-- Q8 grading
|   +-- Q15 temperature
|   +-- Q19 positive / negative / zero
|
+-- ACCESS / WORKFLOW
|   +-- Q11 username + password
|   +-- Q12 age + ID
|   +-- Q17 login + admin
|   +-- Q18 all subjects passed
|
+-- COMPACT DECISION / SIMULATION
|   +-- Q16 ticket type via ternary
|   +-- Q20 ATM PIN + balance workflow
```

Coverage

The supplied practice set repeatedly combines Session 1 values with Session 2 operators and Session 3 branching. That repeated combination is the practical core of the three-session block.

Cross-Session Logic Patterns

Reusable problem-solving templates from Sessions 1-3

PROBLEM-SOLVING ASCII MAP

REUSABLE LOGIC TEMPLATES

```
|
+-- RANGE CHECK
|   value >= min && value <= max
|
+-- MULTI-CONDITION
|   conditionA && conditionB
|
+-- ALTERNATIVE CONDITION
|   conditionA || conditionB
|
+-- DIVISIBILITY
|   value % divisor === 0
|
+-- SIGN CHECK
|   value > 0 / value < 0 / otherwise zero
|
+-- CATEGORY CHAIN
|   if -> else if -> else
|
+-- WORKFLOW GATE
|   validate -> authenticate -> authorize -> act
|
+-- ARRAY EXTREMA
|   Math.max(...array) / Math.min(...array)
```

Session connection

These templates are not separate topics: each one combines concepts from more than one session. They are the bridge from syntax practice to applied programming logic.

Master Syntax Tree

A compact reference for the exact building blocks introduced so far

MASTER ASCII TREE

JAVASCRIPT SYNTAX CORE

```
|
+-- VALUES
|   +-- "text" | 10 | true | null | 10n
|   +-- [1,2,3] | { key: value }
|
+-- VARIABLES
|   +-- var name = value
|   +-- let name = value
|   +-- const name = value
|
+-- OPERATORS
|   +-- arithmetic / assignment / comparison / logical
|   +-- ternary
|
+-- BUILT-INS
|   +-- console.log()
|   +-- prompt()
|   +-- Number()
|   +-- typeof
|   +-- Math.*
|
+-- DATA EXPANSION
|   +-- ...array
|
+-- CONTROL FLOW
|   +-- if / else if / else
```

Quick Revision - Session 1

The essentials to recall before moving into operators

SESSION 1 CHEAT SHEET

SESSION 1 QUICK MAP

`var` -> redeclare + reassign
`let` -> reassign
`const` -> initialize immediately; no reassignment

declaration -> create the variable name
initialization -> give the first value
redeclaration -> declare the same name again
reassignment -> change the current value

Primitive: String / Number / Boolean / Undefined / Null / BigInt
Structured: Array / Object

`prompt()` -> input text
`Number()` -> numeric conversion
`console.log()` -> output
`typeof` -> inspect type

Session 1 in one sentence

Create values, understand their types, store them with variables, accept input, convert it when necessary, and inspect or print the result.

Quick Revision - Session 2

The operator toolkit

SESSION 2 CHEAT SHEET

SESSION 2 QUICK MAP

Arithmetic: + - * / % **

Mutation: ++ --

Assignment: = += -= *= /=

Comparison: == === != !== > < >= <=

Logical: && || !

Ternary: condition ? a : b

Pre -> change first, then use

Post -> use first, then change

== -> loose equality with possible coercion

=== -> strict equality without type conversion

&& -> all required

|| -> at least one

! -> reverse Boolean result

Session 2 in one sentence

Turn values into expressions, transform numbers, compare values, combine conditions, and choose between outcomes.

Quick Revision - Session 3

Reusable tools and branching logic

SESSION 3 CHEAT SHEET

SESSION 3 QUICK MAP

`Math.random()` -> random decimal in `[0,1)`

`Math.floor()` -> round down

`Math.sqrt()` -> square root

`Math.pow()` -> power

`Math.min()` -> minimum argument

`Math.max()` -> maximum argument

`...array` -> expand elements into function arguments

`if` -> test a condition

`else if` -> test another branch

`else` -> fallback branch

Practice patterns -> validation / divisibility / category / access / workflow

Session 3 in one sentence

Use built-in math utilities, expand array values when a function expects separate arguments, and apply conditional control flow to real decision rules.

Final Consolidated Map

The three sessions as one coherent JavaScript foundation

FINAL ASCII ARCHITECTURE

JAVASCRIPT FOUNDATION: SESSIONS 1-3

```
|
+-- 01 REPRESENT
|   values -> variables -> data types -> arrays / objects
|
+-- 02 RECEIVE / INSPECT
|   prompt() -> Number() -> console.log() -> typeof
|
+-- 03 OPERATE
|   arithmetic -> assignment -> Math -> spread
|
+-- 04 COMPARE
|   equality -> relational -> inequality
|
+-- 05 COMBINE
|   && -> || -> !
|
+-- 06 CHOOSE
|   ternary -> if / else if / else
|
+-- 07 VALIDATE / CLASSIFY / CONTROL
|   ranges -> divisibility -> grades -> access -> workflows
|
+-- 08 PRODUCE RESULT
|   console.log() -> visible behavior
```

Three-session foundation

Session 1 teaches what the data is. Session 2 teaches what can be done with the data. Session 3 teaches how reusable tools and control flow can turn those operations into program behavior.

Scope and Coverage Notes

What this document intentionally does and does not add

BOUNDARY MAP

COVERAGE BOUNDARY

INCLUDED FROM SESSIONS 1-3

- +-- Variables: var / let / const
- +-- Declaration / initialization / redeclaration / reassignment
- +-- Primitive and reference/object examples taught
- +-- prompt / Number / console.log / typeof
- +-- Arithmetic / assignment / comparison / logical / ternary
- +-- Math.random / floor / sqrt / pow / min / max
- +-- Spread with arrays and function arguments
- +-- if / else if / else and validation/workflow practice

NOT ADDED AS SESSION CONTENT

- +-- Symbol
- +-- hoisting
- +-- Temporal Dead Zone
- +-- advanced scope rules
- +-- advanced object / array methods

Reason: these topics are not present in the supplied Session 1-3 teaching material.

Use of the reference

The supplied reference tree is treated as a structural inspiration only. This document reorganizes the actual three-session content into a layered architecture and keeps the session boundary explicit.

Final One-Page ASCII Revision

A last-look map for quick review

FINAL REVISION TREE

JAVASCRIPT S1-3

```
|
+-- S1: STATE
|   +-- variables
|   +-- data types
|   +-- arrays / objects
|   +-- input / output
|
+-- S2: OPERATE
|   +-- arithmetic / assignment
|   +-- comparison / logical
|   +-- ternary
|
+-- S3: CONTROL + TOOLS
|   +-- Math methods
|   +-- spread
|   +-- if / else if / else
|   +-- validation / workflow practice
|
+-- COMBINED LOOP
    input -> store -> transform -> compare -> decide -> output
```

Remember the structure

The most useful way to see these sessions is as one chain: Session 1 establishes values and data, Session 2 supplies operators and expressions, and Session 3 adds reusable utilities plus multi-branch control flow.